

ESP32 핀볼 시스템

P1 & P2 전체 코드 명세서

하드웨어 완벽 보정 및 데이터 정밀 전송 메커니즘

Arduino IDE / FreeRTOS Multi-Core

PART 1. 플레이어 1 (P1 마스터)

네트워크 인프라 구축 및 시스템 세팅 제어 전반

P1: 라이브러리 및 통신망 기틀

```
#include // 와이파이 연결 도구
#include // UDP 무선 전송 도구
#include // I2C 통신선 도구
#include // 디스플레이 제어 도구

const char* ssid    = "fusion"; // 공유기 이름
const char* password = "12345678"; // 공유기 비번

// 고정 IP 네트워크 정보 할당
IPAddress local_IP(192, 168, 0, 200); // 본인 보드 주소
IPAddress gateway(192, 168, 0, 1);    // 게이트웨이 주소
IPAddress subnet(255, 255, 255, 0);   // 서브넷 마스크
IPAddress serverIP(192, 168, 0, 138); // 메인 파이썬 PC 주소

const int SEND_PORT    = 10001; // 데이터 나가는 문
const int RECEIVE_PORT = 10003; // 데이터 들어오는 문
WiFiUDP udp; // UDP 객체 생성
```

준비

네트워크 통신과 LCD 전광판 구동에 필요한 무선 통신용 부품
도구들을 소환함

지정

마스터 보드 전용 주소(200번 방)와 데이터를 받아줄 PC 주소
(138번 방)를 유일하게 고정함

P1: 하드웨어 핀 및 상수 제어 정의

```
const int PIN_LR = 35; // 조이스틱 가로 아날로그 핀
const int PIN_UD = 32; // 조이스틱 세로 아날로그 핀
const int SW      = 14; // 조이스틱 내장 스위치 디지털 핀

LiquidCrystal_I2C lcd(0x27, 16, 2); // LCD 주소 및 규격 설정

#define JOY_MID      2048 // 수학적 가운데 기준값
#define JOY_MAX      4095 // 12비트 아날로그 최대값
#define JOY_THRESH   800 // 메뉴 판정 입력 임계값
#define SEND_INTERVAL_MS 10 // 전송 타이밍 주기 (0.01초)
#define CHANGE_THRESH 8 // 송출을 결정하는 최소 변동폭
#define FILTER_SIZE   8 // 이동평균 필터 샘플 크기
#define JOY_DEADZONE 150 // 미세 움직임 무시 안전구역
```

연결

조이스틱의 신호선과 스위치 선을 보드의 물리 핀 번호와 1:1로 매칭시켜 줌

설정

딜레이 시간, 떨림 제한 마진, 중앙 전압 해상도 표준값을 명확한 상수로 선언함

P1: 필터 구조체 및 시스템 상태 관리

```
struct AxisFilter {  
    int buf[FILTER_SIZE]; // 최근 데이터 저장 공간  
    long sum;             // 저장된 값들의 합산 기록  
    int idx;              // 순환 인덱스 포인터  
  
    void init(int pin) {  
        int v = analogRead(pin); sum = 0;  
        for (int i = 0; i < FILTER_SIZE; i++) { buf[i] = v; sum += v; }  
        idx = 0;  
    }  
    int update(int pin) {  
        sum -= buf[idx]; buf[idx] = analogRead(pin); sum += buf[idx];  
        idx = (idx + 1) % FILTER_SIZE;  
        return (int)(sum / FILTER_SIZE); // 실시간 평균값 반환  
    }  
};  
  
AxisFilter filterLR; // 좌우 필터 객체 생성  
AxisFilter filterUD; // 상하 필터 객체 생성  
  
int RAW_LR_MID = 2048; // 초기 영점 임시 저장소  
int RAW_UD_MID = 2048;  
  
enum GameState { STATE_END, STATE_SET, STATE_SRT, STATE_STP };
```

필터

조이스틱 전압이 혼자 부르르 떨리는 현상을 최근 8개 데이터의 실시간 평균값으로 다듬어 줌

상태

대기, 세팅, 경기 시작 등 게임 흐름 변화를 멀티코어가 즉시 인지하도록 휘발성 변수로 선언함

P1: 동기화 오브젝트 및 비상벨 인터럽트

```
// 멀티코어 LCD 자원 통제 신호등 생성
SemaphoreHandle_t lcdSemaphore = NULL;
volatile bool lcdPending = false;
char lcdBuf0[17] = "          ";
char lcdBuf1[17] = "          ";

volatile bool buttonClicked = false;
volatile unsigned long lastInterruptTime = 0;
#define DEBOUNCE_TIME_MS 250

unsigned long tlastSend = 0;
int lastSentX = JOY_MID;
int lastSentY = JOY_MID;

TaskHandle_t NetworkRxTask; // 무선 전담 태스크 핸들러

void IRAM_ATTR clickButton() {
    unsigned long now = millis();
    // 0.25초 이내 다다닥 연타 입력 뺌
    if (now - lastInterruptTime > DEBOUNCE_TIME_MS) {
        buttonClicked = true;
        lastInterruptTime = now;
    }
}
```

통제

두 코어가 한 화면에 글을 적어 글자가 깨지는 문제를 막기 위해
차단용 신호등 틀을 잡음

인터럽트

시스템이 아무리 복잡한 계산 중이라도 버튼이 눌리면 즉각 감지해
0.25초 쿨타임을 먹임

P1: 물리 보정 계산 및 하드웨어 거울 반전

```
int mapAxis(int raw, int midVal) {
    // 데드존 범위는 무조건 완벽 정중앙 처리
    if (abs(raw - midVal) <= JOY_DEADZONE) return JOY_MID;

    int mapped;
    // 실속 중립 전압 기준 비례 스케일 배분
    if (raw < midVal) mapped = map(raw, 0, midVal - JOY_DEADZONE, 0, JOY_MID);
    else mapped = map(raw, midVal + JOY_DEADZONE, JOY_MAX, JOY_MID, JOY_MAX);

    return constrain(mapped, 0, JOY_MAX);
}

int getFilteredX() {
    int raw = filterLR.update(PIN_LR);
    int val = mapAxis(raw, RAW_LR_MID);
    int x = JOY_MAX - val; // 반대로 뒤집혀 쏘힌 부품 방향 반전 보정
    if (abs(x - JOY_MID) <= 100) x = JOY_MID;
    return x;
}

int getFilteredY() {
    int raw = filterUD.update(PIN_UD);
    return mapAxis(raw, RAW_UD_MID);
}
```

교정

조이스틱 부품마다 빼돌어진 원래의 물리적 가운데 중심점을
수학적인 2048 센터로 원복함

반전

회로가 좌우 거꾸로 장착된 환경을 코딩단에서 수학적으로
상하좌우를 매끄럽게 뒤집어 줌

P1: 신호등 기반 LCD 문자열 출력 제어

```
void requestLCD(const char* text) {
    if (lcdSemaphore == NULL) return;
    // 신호등이 초록불(열쇠 획득)일 때만 버퍼에 글을 씀
    if (xSemaphoreTake(lcdSemaphore, pdMS_TO_TICKS(10)) == pdTRUE) {
        size_t len = strlen(text);
        if (len <= 16) {
            snprintf(lcdBuf0, 17, "%-16s", text); snprintf(lcdBuf1, 17, "%-16s", "");
        } else {
            char line0[17], line1[17]; strncpy(line0, text, 16); line0[16] = '\0'; strncpy(line1, text, 16); line1[16] = '\0';
            snprintf(lcdBuf0, 17, "%-16s", line0); snprintf(lcdBuf1, 17, "%-16s", line1);
        }
        lcdPending = true;
        xSemaphoreGive(lcdSemaphore); // 일 끝나면 바로 신호등 반납
    }
}

void renderLCD() {
    if (!lcdPending || lcdSemaphore == NULL) return;
    char line0[17], line1[17];
    if (xSemaphoreTake(lcdSemaphore, pdMS_TO_TICKS(10)) == pdTRUE) {
        memcpy(line0, lcdBuf0, 17); memcpy(line1, lcdBuf1, 17);
        lcdPending = false;
        xSemaphoreGive(lcdSemaphore);
    } else return;
}
```

적재

출력할 문자열을 한 글자씩 쪼개 임시 보관 버퍼에 담아 안전 마진을 마련함

출력

신호등 통제를 받아 버퍼에 담긴 정돈된 데이터를 노이즈 없이 LCD 소자에 깨끗하게 밀어 넣음

P1: 무선 송신 및 Core 0번 수신 루프

```
void sendUDP(const String& msg) {
    if (WiFi.status() == WL_CONNECTED) {
        udp.beginPacket(serverIP, SEND_PORT); udp.print(msg); udp.endPacket();
    }
}

void networkRxLoop(void* pvParameters) {
    char rxBuffer[255];
    for (;;) { // 무한 감시 루프 가동
        if (WiFi.status() == WL_CONNECTED) {
            int packetSize = udp.parsePacket();
            if (packetSize) {
                int len = udp.read(rxBuffer, sizeof(rxBuffer) - 1); if (len > 0) rxBuffer[len] = '\0';
                String response = String(rxBuffer); response.trim();

                if (response.indexOf('.') != -1 && response.indexOf("END") != -1) {
                    gState = STATE_END; joystickActive = false; resetMenuFlags = true;
                    char part1[10], part2[10], part3[10]; sscanf(response.c_str(), "%[^:]:%[^:]:%[^:]",
                        requestLCD(("You " + String(part2)).c_str()); // 두번째 마스터 결과 출력
                    continue;
                }
                else if(response == "STP") { gState = STATE_STP; joystickActive = false; resetMenuFlags = true;
                else if(response == "SET") { gState = STATE_SET; joystickActive = false; resetMenuFlags = true;
                else if(response == "SRT") { gState = STATE_SRT; joystickActive = true; resetMenuFlags = true;
```

송신

내 조이스틱 좌표 수치를 무선 통신망 규격에 맞춰 서버 컴퓨터로
고속 업로드함

수신

0번 머리가 온종일 대기하며 컴퓨터가 보내오는 시작, 종료, 세팅
무선 지령을 즉각 가로챈

P1: 시스템 오토 캘리브레이션 및 하드웨어 빌드Up

```
void setup() {
    Serial.begin(115200);

    lcdSemaphore = xSemaphoreCreateBinary(); xSemaphoreGive(lcdSemaphore); // 신호등 초기 생성
    Wire.begin(25, 26); lcd.init(); lcd.backlight(); lcd.clear();

    WiFi.config(local_IP, gateway, subnet); WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) { delay(200); } // 연결 대기 연출

    analogReadResolution(12); // 아날로그 정밀도 12비트 확장 적용
    pinMode(SW, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(SW), clickButton, FALLING);
    udp.begin(RECEIVE_PORT);

    // 초기 전압 64번 더해 평균내는 자동 영점 교정
    long sumLR = 0, sumUD = 0;
    for (int i = 0; i < 64; i++) { sumLR += analogRead(PIN_LR); sumUD += analogRead(PIN_UD); }
    RAW_LR_MID = (int)(sumLR / 64); RAW_UD_MID = (int)(sumUD / 64);

    filterLR.init(PIN_LR); filterUD.init(PIN_UD);

    // Core 0번에 무선 패킷 청취 전담 업무 강제 지정 배치
    xTaskCreatePinnedToCore(networkRxLoop, "NetworkRxTask", 10000, NULL, 1, &NetworkRxTask, 0);
    requestLCD("P1 CONTROLLER");
}
```

정비

부팅 순간의 고유한 아날로그 전압을 64번 샘플링 연산하여 기계 오차를 깨끗이 닦아냄

배정

듀얼코어의 0번 머리에 무선 네트워크 편지함을 감시하는 고정 임무를 강제 부여함

P1: 메인 제어 루프 및 주도권 신호 핸들링

```
void loop() {  
    renderLCD(); // 화면 주기적 새로고침  
  
    if (buttonClicked) {  
        buttonClicked = false;  
        if (gState == STATE_END || gState == STATE_SRT || gState == STATE_STP) sendUDP("SET\\n");  
        else if (gState == STATE_SET) sendUDP("CLK\\n"); // 세팅 진입 주도권  
    }  
  
    static bool lastUpState = false; static bool lastDnState = false;  
    if (resetMenuFlags) { lastUpState = false; lastDnState = false; resetMenuFlags = false; }  
  
    int cx = getFilteredX(); int cy = getFilteredY();  
  
    if (gState == STATE_END) { // 스틱 뺏겨서 게임 기선제압 시작 요청  
        bool isUp = (cy < JOY_MID - JOY_THRESH);  
        if (isUp && !lastUpState) sendUDP("SRT\\n");  
        lastUpState = isUp;  
    }  
    else if (gState == STATE_SET) { // 설정값 위아래 조작 제어 신호  
        bool isUp = (cy < JOY_MID - JOY_THRESH); bool isDn = (cy > JOY_MID + JOY_THRESH);  
        if (isUp && !lastUpState) sendUDP("UP\\n"); if (isDn && !lastDnState) sendUDP("DN\\n");  
        lastUpState = isUp; lastDnState = isDn;  
    }  
}
```

조작

대기 화면에서 스틱을 위로 올리면 게임이 켜지고 메뉴를 바꾸는 등 주도적 제어를 행사함

개방

영역 브레이크 한계를 걸지 않고 패들 전 범위를 컴퓨터로 던져 자유로운 돌파 축구를 실현함

PART 2. 플레이어 2 (P2 게스트)

독립적인 독자 포트 할당 및 게스트 맞춤형 안전 연산 로직

P2: 독립 수신 포트 및 네트워크 할당

```
#include
#include
#include
#include

const char* ssid      = "fusion";
const char* password = "12345678";

// P2 보드 고유 번호 설정
IPAddress local_IP(192, 168, 0, 154); // P2 게스트 전용 룸
IPAddress gateway(192, 168, 0, 1);
IPAddress subnet(255, 255, 255, 0);
IPAddress serverIP(192, 168, 0, 138);

const int SEND_PORT    = 10002; // P1과 안 겹치게 비켜 간 출구
const int RECEIVE_PORT = 10004; // P1과 충돌 없는 전용 입구

WiFiUDP udp;
```

지정

P1 마스터와 주소가 꼬이지 않게 게스트 전용 IP(154번 방)를
유니크하게 매립함

교차

송신 문(10002번)과 수신 문(10004번)을 서로 다르게 교차
구성하여 유선 신호 간섭을 차단함

P2: 필터 및 인터럽트 시스템 구조체 빌드

```
const int PIN_LR = 35;
const int PIN_UD = 32;
const int SW      = 14;

LiquidCrystal_I2C lcd(0x27, 16, 2);

#define JOY_MID      2048
#define JOY_MAX      4095
#define JOY_THRESH   800
#define SEND_INTERVAL_MS 10
#define CHANGE_THRESH 8
#define FILTER_SIZE   8
#define JOY_DEADZONE 150

struct AxisFilter {
    int buf[FILTER_SIZE]; long sum; int idx;
    void init(int pin) {
        int v = analogRead(pin); sum = 0; for (int i = 0; i < FILTER_SIZE; i++) { buf[i] = v; sum
    }
    int update(int pin) {
        sum -= buf[idx]; buf[idx] = analogRead(pin); sum += buf[idx]; idx = (idx + 1) % FILTER_SIZE;
        return (int)(sum / FILTER_SIZE);
    }
};
```

매핑

가로, 세로, 내부 푸시 버튼 아날로그 데이터 노선을 하드웨어 칩핀
배열에 정확하게 올려놓음

평균

8칸 회전 초밥식 원형 큐 상자를 동일하게 구현하여 수치 흔들림
현상을 짝 붙잡음

P2: 동기화 전역 변수 세팅 및 인터럽트 가동

```
AxisFilter filterLR; AxisFilter filterUD;
int RAW_LR_MID = 2048; int RAW_UD_MID = 2048;

enum GameState { STATE_END, STATE_SET, STATE_SRT, STATE_STP };
volatile GameState gState = STATE_END;
volatile bool joystickActive = false;
volatile bool resetMenuFlags = false;

SemaphoreHandle_t lcdSemaphore = NULL;
volatile bool lcdPending = false;
char lcdBuf0[17] = " "; char lcdBuf1[17] = " ";

volatile bool buttonClicked = false;
volatile unsigned long lastInterruptTime = 0;
#define DEBOUNCE_TIME_MS 250

unsigned long tlastSend = 0; int lastSentX = JOY_MID; int lastSentY = JOY_MID;
TaskHandle_t NetworkRxTask;

void IRAM_ATTR clickButton() {
    unsigned long now = millis();
    if (now - lastInterruptTime > DEBOUNCE_TIME_MS) { buttonClicked = true; lastInterruptTime = now; }
}
```

정렬

수신 전용 코어 테스트와 메인 조작 연산 변수의 동기화 보정용
변수들을 빠짐없이 깔아둠

방지

버튼 점점 편차가 가치는 연타 노이즈를 0.25초 디바운스 쿨타임을
걸어 완벽 차단해 둠

P2: 축 보정 캘리브레이션 및 버퍼 전광판 통제

```
int mapAxis(int raw, int midVal) {
    if (abs(raw - midVal) <= JOY_DEADZONE) return JOY_MID;
    int mapped;
    if (raw < midVal) mapped = map(raw, 0, midVal - JOY_DEADZONE, 0, JOY_MID);
    else mapped = map(raw, midVal + JOY_DEADZONE, JOY_MAX, JOY_MID, JOY_MAX);
    return constrain(mapped, 0, JOY_MAX);
}

int getFilteredX() {
    int raw = filterLR.update(PIN_LR); int val = mapAxis(raw, RAW_LR_MID);
    int x = JOY_MAX - val; // 반사 거울 모드 화면 매칭 처리
    if (abs(x - JOY_MID) <= 100) x = JOY_MID;
    return x;
}

int getFilteredY() {
    int raw = filterUD.update(PIN_UD); return mapAxis(raw, RAW_UD_MID);
}
```

매칭

데드존 안전 가드 범위를 반영해서 정밀 조이스틱 눈금을 0에서 4095 폴 스케일로 바꿈

동기화

상대편 시점과 똑같은 궤적으로 패들이 전진하도록 좌표 대칭 상을 유연하게 일치시킴

P2: 상호 통제 신호등 및 유선 주사 핸들러

```
void requestLCD(const char* text) {
    if (lcdSemaphore == NULL) return;
    // 신호등이 초록불일 때 안전하게 글쓰기 진입
    if (xSemaphoreTake(lcdSemaphore, pdMS_TO_TICKS(10)) == pdTRUE) {
        size_t len = strlen(text);
        if (len <= 16) { snprintf(lcdBuf0, 17, "%-16s", text); snprintf(lcdBuf1, 17, "%-16s", "")
        } else {
            char line0[17], line1[17]; strncpy(line0, text, 16); line0[16] = '\0'; strncpy(line1, text + 16, 16); line1[16] = '\0';
            snprintf(lcdBuf0, 17, "%-16s", line0); snprintf(lcdBuf1, 17, "%-16s", line1);
        }
        lcdPending = true; xSemaphoreGive(lcdSemaphore); // 일 끝나면 바로 신호등 반납
    }
}

void renderLCD() {
    if (!lcdPending || lcdSemaphore == NULL) return;
    char line0[17], line1[17];
    if (xSemaphoreTake(lcdSemaphore, pdMS_TO_TICKS(10)) == pdTRUE) {
        memcpy(line0, lcdBuf0, 17); memcpy(line1, lcdBuf1, 17); lcdPending = false; xSemaphoreGive(lcdSemaphore);
    } else return;
    lcd.clear(); lcd.setCursor(0, 0); lcd.print(line0); lcd.setCursor(0, 1); lcd.print(line1);
}

void sendUDP(const String& msg) {
```

통제

멀티코어가 공유 자원인 LCD 전광판을 동시에 치고 들어가 하드웨어가 꼬이는 대참사를 막아둠

주사

메인 테스트 안에서만 실제 클리어 명령과 커서 주사를 단독 실행해 안전성을 확보함

P2: 3번 토막 데이터 독립 파싱 수신부

```
void networkRxLoop(void* pvParameters) {
    char rxBuffer[255];
    for (;;) {
        if (WiFi.status() == WL_CONNECTED) {
            int packetSize = udp.parsePacket();
            if (packetSize) {
                int len = udp.read(rxBuffer, sizeof(rxBuffer) - 1); if (len > 0) rxBuffer[len] = '\0';
                String response = String(rxBuffer); response.trim();

                if (response.indexOf(':') != -1 && response.indexOf("END") != -1) {
                    gState = STATE_END; joystickActive = false; resetMenuFlags = true;
                    char part1[10], part2[10], part3[10]; sscanf(response.c_str(), "%[^:]:%[^:]:%[^:]",
                        // 게스트 맞춤형 핵심 차별점: 3번째 데이터 조각(P2 전용 승패) 추출
                        requestLCD(("You " + String(part3)).c_str());
                    continue;
                }
                else if(response == "STP") { gState = STATE_END; joystickActive = false; resetMenuFlags = true;
                else if(response == "SET") { gState = STATE_SET; joystickActive = false; resetMenuFlags = true;
                else if(response == "SRT") { gState = STATE_SRT; joystickActive = true; resetMenuFlags = true;
            }
        }
        vTaskDelay(1 / portTICK_PERIOD_MS);
    }
}
```

파싱

컴퓨터가 날린 전체 경기 승패 문자열 데이터 덩어리를 콜론(:)
단위로 정교하게 분리해 냄

예약

세 번째 위치에 쪼개진 나만의 결과 데이터 파트를 낚아채서 내
전광판 화면에 예약 표출함

P2: 인프라 부트스랩 및 비상 탈주 매커니즘 마감

```
void setup() {
    Serial.begin(115200);
    lcdSemaphore = xSemaphoreCreateBinary(); xSemaphoreGive(lcdSemaphore);
    Wire.begin(25, 26); lcd.init(); lcd.backlight(); lcd.clear();

    WiFi.config(local_IP, gateway, subnet); WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) { delay(200); }

    analogReadResolution(12); pinMode(SW, INPUT_PULLUP); attachInterrupt(digitalPinToInterrupt(
    udp.begin(RECEIVE_PORT);

    long sumLR = 0, sumUD = 0;
    for (int i = 0; i < 64; i++) { sumLR += analogRead(PIN_LR); sumUD += analogRead(PIN_UD); de
    RAW_LR_MID = (int)(sumLR / 64); RAW_UD_MID = (int)(sumUD / 64);

    filterLR.init(PIN_LR); filterUD.init(PIN_UD);
    xTaskCreatePinnedToCore(networkRxLoop, "NetworkRxTask", 10000, NULL, 1, &NetworkRxTask, 0);
    requestLCD("P2 CONTROLLER");
}

void loop() {
    renderLCD();
    if (buttonClicked) {
        buttonClicked = false;
    }
}
```

구동

필터 가동 및 전담 수신 루프를 독립 코어에 할당해 게임 하드웨어 시스템을 준비함

탈출

게임 도중 불리하거나 비상 상황일 때, P2가 버튼을 누르면 즉시 판을 터뜨리는 파괴 신호를 전송함

마스터 (P1) vs 게스트 (P2) 아키텍처 비교 요약

비교 분류 항목	P1 마스터 컨트롤 보드 (Master)	P2 게스트 제어 보드 (Guest)
고유 통신 IP 주소	192.168.0.200 (대기방 주도 고정 할당)	192.168.0.154 (게스트 독립 통신 주소)
네트워크 전송 포트	송신: 10001 / 수신: 10003	송신: 10002 / 수신: 10004
시스템 제어 특권	설정 모드 전환(SET) 및 데이터 선택/결정(CLK)	경기 진행 중 긴급 전면 폭파 강제 종료(END)
데이터 수신 처리 요령	파싱된 무선 편지의 2번째 토막(P1 결과) 매립	파싱된 무선 편지의 3번째 토막(P2 결과) 매립
동기화 안전 장치	이진 세마포어(Semaphore) 신호등 체계를 통한 듀얼코어 LCD 간섭 대참사 방지 공동 내장	